Lab 2: Memory Hierarchy

Set associativity, replacement, and a two-level design search

Maxwell Wu (mwu3) | October 25, 2026

Overview and evidence status

The simulator models a parameterized data-cache hierarchy driven by Pin memory references. The completed Task 4 search evaluates 20 distinct hierarchies on the SPEC2017 `602.gcc_s` and `641.leela_s` test workloads, producing 40 benchmark measurements and 18 distinct Destiny cache models. The best measured hierarchy is an 8 KiB, 64-byte, 4-way LRU L1 and a 512 KiB, 128-byte, 8-way LRU L2. Its equally weighted mean AMAT is 0.25416 ns, modeled area is 6.26309 mm^2, and estimated total cache power is 152.72 mW at an assumed 10^9 L1 accesses/s. This is the best design in the measured set, not a demonstrated global optimum.

The corrected single-level collection adds 27 distinct configurations and 54 benchmark measurements: 18 LRU geometries for Task 1 and four policies at each of three geometries for Task 2. Three LRU configurations are shared by the tasks. The report uses those actual single-level executions; legacy tree-PLRU results are excluded from the experimental comparisons.

Table 1: Evidence available for the handout's requirements.

Task	Evidence in this report	Qualification
1	18 LRU geometries on both benchmarks; miss-rate and AMAT plots	Single-level runs with Destiny latencies.
2	Four policies at three geometries on both benchmarks	One run per case; one random seed.
3	Parameterized hierarchy and deterministic tests	Demand-only model; no writeback or coherence.
4	20 designs, Destiny metrics, Pareto and sensitivity plots	Bounded search; one run per benchmark/design.

Workloads and provenance

The completed session is `session-3973386ea9622d3cee1b`, under `results/task4/`. Its manifest records commands, inputs, source and tool hashes, and environment. Gcc compiles `t1.c` with `-O3 -finline-limit=50000`; leela runs `test.sgf`. Each is the sole command in its selected base/test run directory. Pin instruments the full process, including library references; there is no instruction-limit sampling or warmup. Runs use one OpenMP thread, disabled ASLR (successful in this session), and a deterministic random-replacement seed. Eight designs were reused after provenance validation and twelve were newly evaluated. Completion was recorded at 18:57:30 Eastern on October 24, 2026.

1 Simulator design: Tasks 1–3

1.1 Set-associative lookup

For capacity C bytes, block size B, and associativity A, the number of sets is $S = C/(BA)$. Address a maps to block $q = \lfloor a/B \rfloor$, set $q \bmod S$, and tag $\lfloor q/S \rfloor$. The implementation requires power-of-two block size, associativity, and set count, with integral capacity division. Each line stores a valid bit and tag. A lookup scans the selected set and fills an invalid way before evicting a valid line. A reference spanning blocks generates one access per touched L1 block. Pin counts a read-modify-write operand once as a read and once as a write, using predicated callbacks.

1.2 Replacement policies

Random.
Fixed-seed xorshift32 selects a victim when the set is full. It ignores reuse, and a single seed does not quantify variability across seeds.

LRU.
A per-set ordered list moves every hit or fill to the most-recent position and evicts the oldest. Recency tracking costs state and update logic.

Bit-PLRU.
A touch sets one bit per way; when all bits are set, all other bits are cleared. The first clear bit identifies a victim. This differs from the tree-PLRU used in the older saved measurements.

Round-robin.
The additional policy uses a per-set pointer that advances on full-set replacement. Its low-state cyclic selection may avoid random imbalance but cannot protect hot blocks based on hits.

1.3 Two-level behavior and boundaries

Each level has independent capacity, block size, associativity, and policy. L2 blocks must be a multiple of, and at least as large as, L1 blocks. L1 misses access L2; L2 misses increment DRAM. Without L2, each L1 miss increments DRAM. Reads and writes allocate on a miss. L2 demand fetches are modeled as reads, including fetches caused by L1 store misses. The model has no dirty bits, writebacks, inclusion invalidations, or writes propagated on L1 hits. It is not a complete write-through or write-back implementation.

Instruction-cache traffic, physical-address translation, prefetching, coherence, queueing, overlap, and memory-level parallelism are excluded. Instrumented virtual addresses drive the model in a single-thread workload setup. AMAT is a serialized access estimate, not CPU execution time or an official SPEC score.

1.4 Validation

`tests/cache_test.cpp` checks LRU victims, bit-PLRU behavior distinct from tree-PLRU, repeatable random replacement, direct-mapped policy equivalence, cross-block references, write hits, single-level DRAM counts, and larger shared L2 blocks. These tests passed on ECE before the Pin rebuild. Collection requires zero process exit status and consistent counters, including $N = H + M$, $N = R + W$, $N_2 = M_1$, and $N_{\text{DRAM}} = M_2$ for two-level runs. It does not perform SPEC reference-output comparison. Python tests cover parsing, AMAT, budgets, 20-design accounting, output generation, and reuse validation using fixture data.

2 Task 1: associativity, block size, and AMAT

With L1 miss rate m_1, Destiny hit latency t_{1h} and miss latency t_{1m}, the handout's single-level model is

$$\mathrm{AMAT}_1 = (1 - m_1)t_{1h} + m_1(t_{1m} + 9.5 \text{ ns}). \tag{1}$$

The single-level sweep crosses capacities 2, 4, and 8 KiB, block sizes 32 and 64 bytes, and associativities 1, 2, and 4, with LRU throughout. Both benchmarks execute at all 18 geometries. Every L1 miss accesses DRAM; Destiny supplies the geometry-specific latency, so AMAT includes the cost of a more complex lookup.

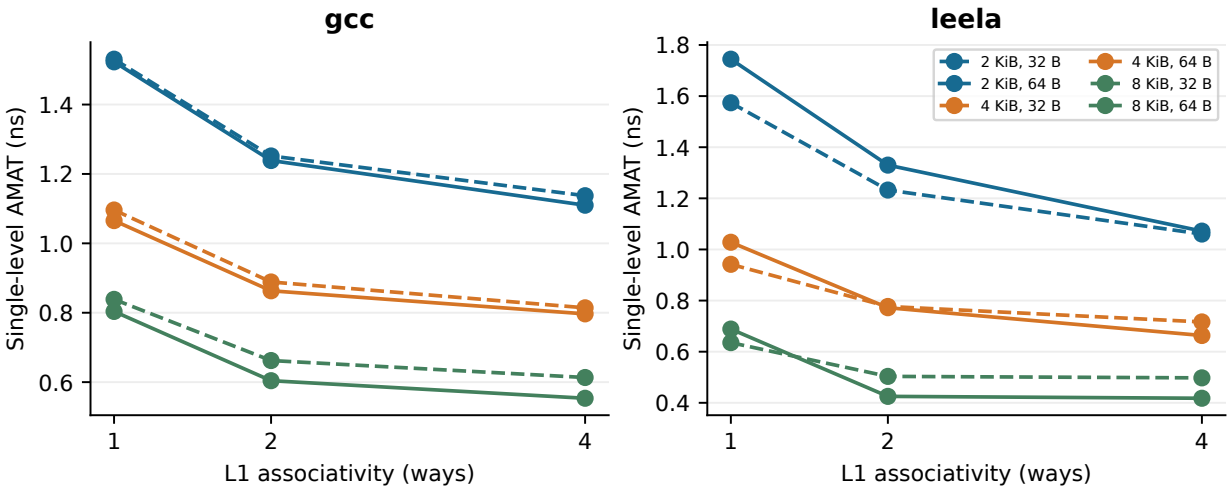


Figure 1: Single-level AMAT from 36 benchmark measurements and Destiny latencies. Dashed lines use 32-byte blocks; solid lines use 64-byte blocks.

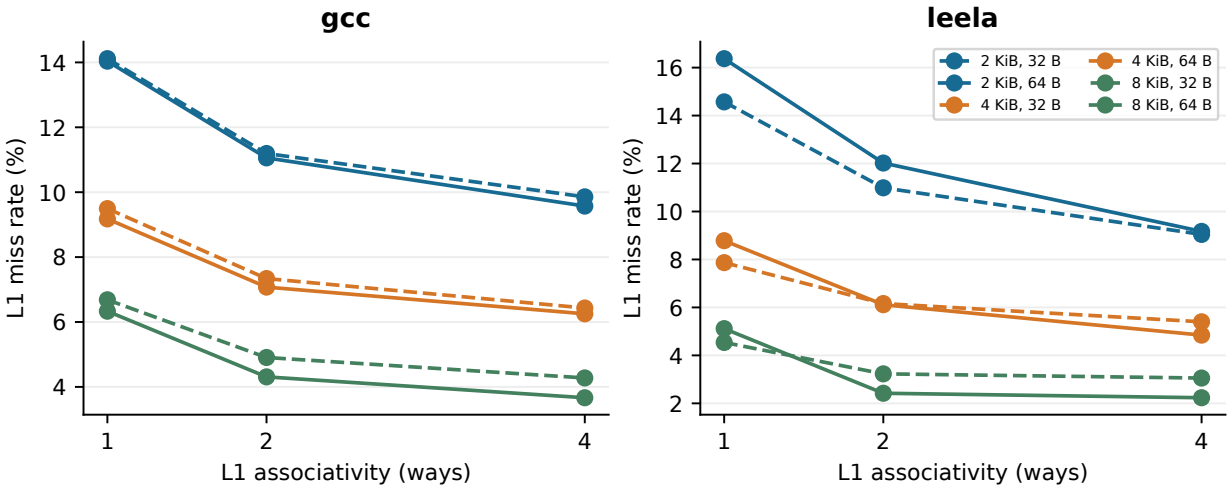


Figure 2: Measured L1 miss rates for the same 18 geometries per benchmark.

At 8 KiB/64 B, increasing associativity from 1 to 4 lowers gcc's miss rate from 6.333% to 3.666% and leela's from 5.116% to 2.234%. AMAT falls from 0.8038 to 0.5535 ns for gcc and from 0.6885 to 0.4178 ns for leela. Most of leela's gain occurs by two ways (0.4252 ns); the next increase produces a smaller benefit. Associativity reduces conflict pressure, but lookup latency must also be counted. At four ways, doubling the block size from 32 to 64 bytes lowers gcc AMAT at all three capacities. Leela instead rises from 1.0603 to 1.0718 ns at 2 KiB, then improves at 4 and 8 KiB. Larger blocks exploit spatial locality but leave fewer resident blocks; their benefit depends on the working set. At 64 B/four ways, increasing capacity from 2 to 8 KiB reduces gcc AMAT from 1.1099 to 0.5535 ns and leela from 1.0718 to 0.4178 ns. These gains describe the measured test streams, not a universal capacity or associativity optimum.

3 Task 2: replacement-policy evaluation

The intended three configurations are 2 KiB/1-way, 4 KiB/2-way, and 8 KiB/4-way, all with 64-byte blocks. They provide a direct-mapped control and increasing capacity/associativity to expose conflict and reuse effects. Since size and ways change together, compare policies *within* a geometry; cross-geometry comparisons do not isolate either parameter.

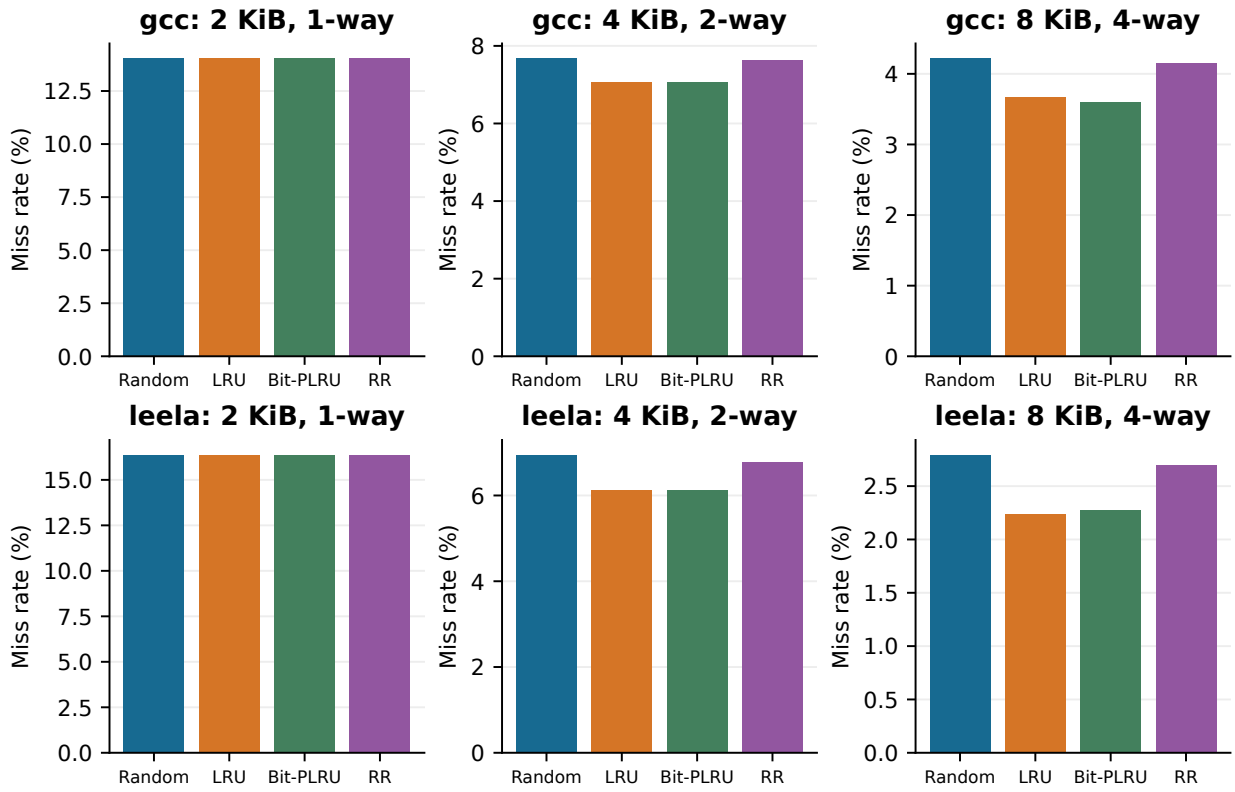


Figure 3: All four policies on all three single-level geometries, with 64-byte blocks. RR denotes round-robin. Each panel compares one fixed geometry and benchmark.

The direct-mapped control gives 14.038% misses for gcc and 16.373% for leela under all four policies at the displayed precision. Gcc's small spread of about 0.000020 percentage points is run variation, not a replacement advantage: there is only one possible victim. Disabling ASLR does not guarantee identical reference streams across separate instrumented executions.

At 4 KiB/2-way, LRU and bit-PLRU are effectively tied: 7.075% misses for gcc and 6.113% for leela. Random gives 7.678% and 6.946%, and round-robin gives 7.622% and 6.763%. Recency protects reused blocks better here. With two ways, the bit-PLRU rule selects the least recently touched way, explaining its close agreement with LRU.

At 8 KiB/4-way, gcc slightly favors bit-PLRU (3.595%) over LRU (3.666%), whereas leela favors LRU (2.234%) over bit-PLRU (2.275%). Random gives 4.219% and 2.793%; round-robin gives 4.145% and 2.691%. Approximate recency can alter eviction decisions in ways that benefit a particular stream; exact LRU is not always the best policy. Round-robin provides a cheap deterministic alternative but ignores hit recency. The full four-policy comparison is shown above; single runs and one random seed do not establish statistical significance or a workload-independent ordering.

4 Task 4: objective and complete design catalog

Let m_2 be the local L2 miss rate, conditional on an L1 miss. The model is

$$\text{AMAT}_2 = (1 - m_1)t_{1h} \tag{2}$$
$$+ m_1 \left[t_{1m} + (1 - m_2)t_{2h} + m_2(t_{2m} + 9.5 \text{ ns})\right]. \tag{3}$$

DRAM costs $19 \times 0.5 = 9.5$ ns. All terms use ns. Ranking minimizes the arithmetic mean of gcc and leela AMAT with equal benchmark weights; exact ties use lower area, then lower estimated power. No equivalence tolerance is assumed.

The search starts with eight L1 geometries and a 512 KiB/64 B/8-way LRU L2. Using the best L1, it adds three matched non-LRU policy pairs, one L2 block-size alternative, four L2 associativities, and four capacities: $8 + 3 + 1 + 4 + 4 = 20$. The sensitivity curves reuse the baseline. Independent level policies are supported by the tool, but this budgeted experiment tests matched pairs only. All designs use at most 4104 KiB of data, below both 5,000,000 bytes and the script's 5 MiB limit. The winner uses 520 KiB.

Table 2: All 20 measured designs. C/B/A/P denotes KiB, block bytes, ways, and policy; L=LRU, R=random, P=bit-PLRU, RR=round-robin. AMAT is in ns, area in mm^2, power in mW.

ID	L1: C/B/A/P	L2: C/B/A/P	AMAT	Area	Power
D01	8/32/1/L	512/64/8/L	0.2941	7.367	153.62
D02	8/32/4/L	512/64/8/L	0.2877	7.089	128.99
D03	8/32/8/L	512/64/8/L	0.3099	7.093	142.95
D04	8/64/1/L	512/64/8/L	0.2928	7.336	205.08
D05	8/64/4/L	512/64/8/L	0.2826	7.049	148.72
D06	8/64/8/L	512/64/8/L	0.3047	7.041	159.10
D07	32/32/1/L	512/64/8/L	0.3197	8.527	200.83
D08	32/32/4/L	512/64/8/L	0.3019	7.639	167.82
D09	8/64/4/R	512/64/8/R	0.2885	7.049	151.68
D10	8/64/4/P	512/64/8/P	0.2832	7.049	148.67
D11	8/64/4/RR	512/64/8/RR	0.2859	7.049	151.14
D12	8/64/4/L	512/128/8/L	0.2542	6.263	152.72
D13	8/64/4/L	512/64/1/L	0.2923	6.784	148.03
D14	8/64/4/L	512/64/2/L	0.2844	7.431	165.11
D15	8/64/4/L	512/64/4/L	0.2818	6.060	145.83
D16	8/64/4/L	512/64/16/L	0.2834	6.483	143.13
D17	8/64/4/L	128/64/8/L	0.2997	2.309	97.56
D18	8/64/4/L	256/64/8/L	0.2887	3.596	113.13
D19	8/64/4/L	1024/64/8/L	0.2833	12.406	199.27
D20	8/64/4/L	4096/64/8/L	0.2941	32.203	436.70

This search emphasizes moderate L1 sizes and reuse of completed measurements to control runtime. Many capacity/block/policy combinations remain unexplored. The recommendation is the best measured candidate, not a global optimum.

5 Pareto tradeoffs and per-benchmark AMAT

A measured design is Pareto optimal if no other design is no worse in mean AMAT, area, and estimated total power and strictly better in at least one. Six designs meet this three-objective criterion. A three-objective Pareto point need not be on both two-dimensional projected frontiers.

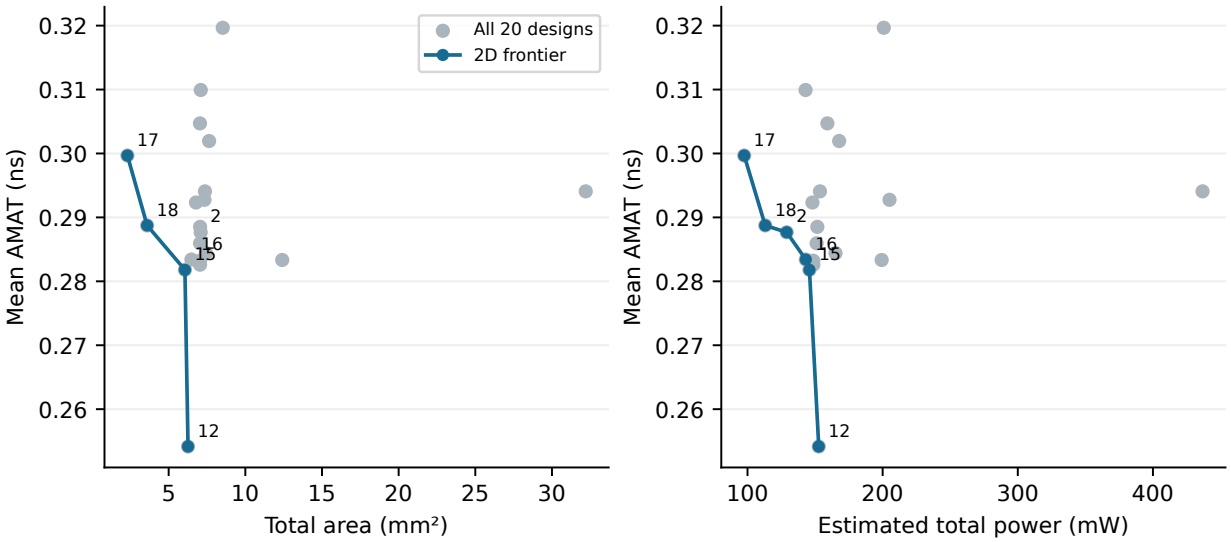


Figure 4: All 20 designs and separate two-dimensional frontiers. Numeric labels identify the six three-objective Pareto designs by Table 2 ID. Lines are visual guides, not interpolated designs.

Table 3: Three-objective Pareto set, ordered by mean AMAT.

ID	AMAT (ns)	Area (mm^2)	Power (mW)
D12	0.2542	6.263	152.72
D15	0.2818	6.060	145.83
D16	0.2834	6.483	143.13
D02	0.2877	7.089	128.99
D18	0.2887	3.596	113.13
D17	0.2997	2.309	97.56

D12 minimizes mean AMAT. Relative to D12, D17 increases AMAT by about 17.9% but reduces area by about 63.1% and power by about 36.1%. D02 offers another low-power tradeoff using a different L1 block size. A stricter physical budget could favor these alternatives; the stated performance-first objective selects D12.

5.1 Per-benchmark results for the Pareto candidates

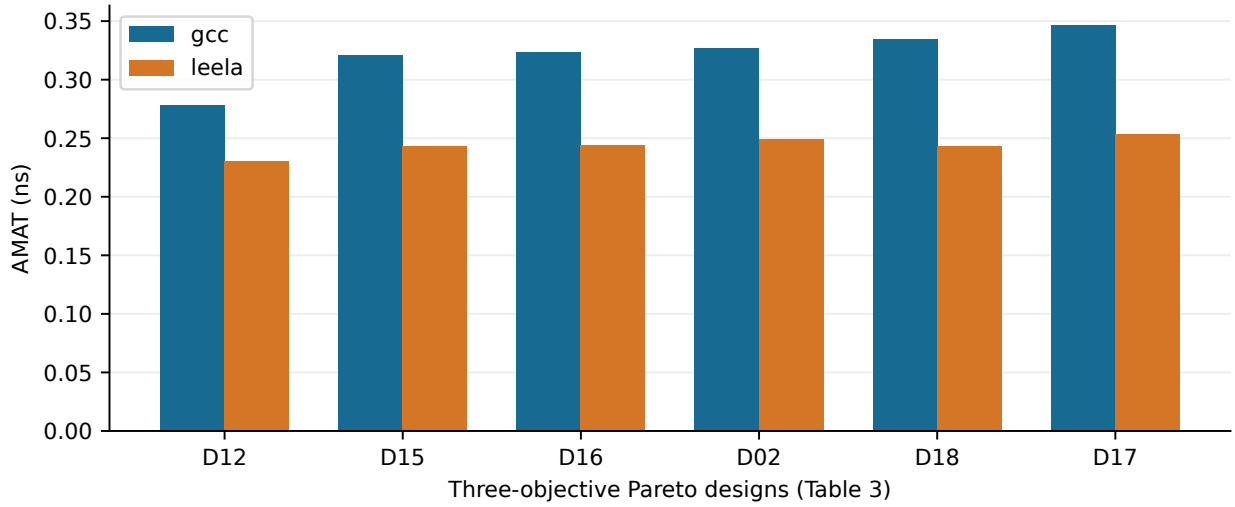


Figure 5: Gcc and leela AMAT for the six three-objective Pareto designs. Bars show workload differences, not statistical uncertainty.

Table 4: D12 benchmark-specific performance. DRAM/L1 is the global memory-access fraction, not the local L2 miss rate.

Benchmark	L1 miss (%)	L2 local miss (%)	DRAM / L1 (%)	AMAT (ns)
gcc	3.666	16.571	0.608	0.2784
leela	2.234	6.831	0.153	0.2299

For the selected hierarchy, gcc has higher L1 and local L2 miss rates than leela, and its AMAT is about 21% higher. The global DRAM fraction is about 0.608% for gcc versus 0.153% for leela. These differences show why both workloads must be measured: an aggregate objective alone can conceal different memory behaviors.

The reported mean is $(0.2784 + 0.2299)/2 \approx 0.2542$ ns, using the unrounded values in the actual calculation. It gives each benchmark equal weight, rather than pooling counters and allowing the longer reference stream to dominate. The bars retain the individual benchmark values for all six Pareto candidates so alternative application priorities can be assessed directly.

Small global DRAM fractions explain AMAT well below 9.5 ns: DRAM latency is paid rarely. This does not imply full instructions execute in fractions of a cycle. The cold-start demand model does not capture overlap or throughput, and AMAT ratios are not runtime speedups. Without repeated runs or multiple random seeds, the plotted differences also provide no statistical confidence bounds.

6 L2 sensitivity and selected hierarchy

The fixed L1 is 8 KiB/64 B/4-way LRU. Both sweeps use 64-byte L2 blocks and LRU. The associativity sweep fixes L2 at 512 KiB; the size sweep fixes it at 8 ways. Their common anchor is D05, not final winner D12 (which has 128-byte L2 blocks). Each curve changes one architectural parameter, recalculating Destiny costs.

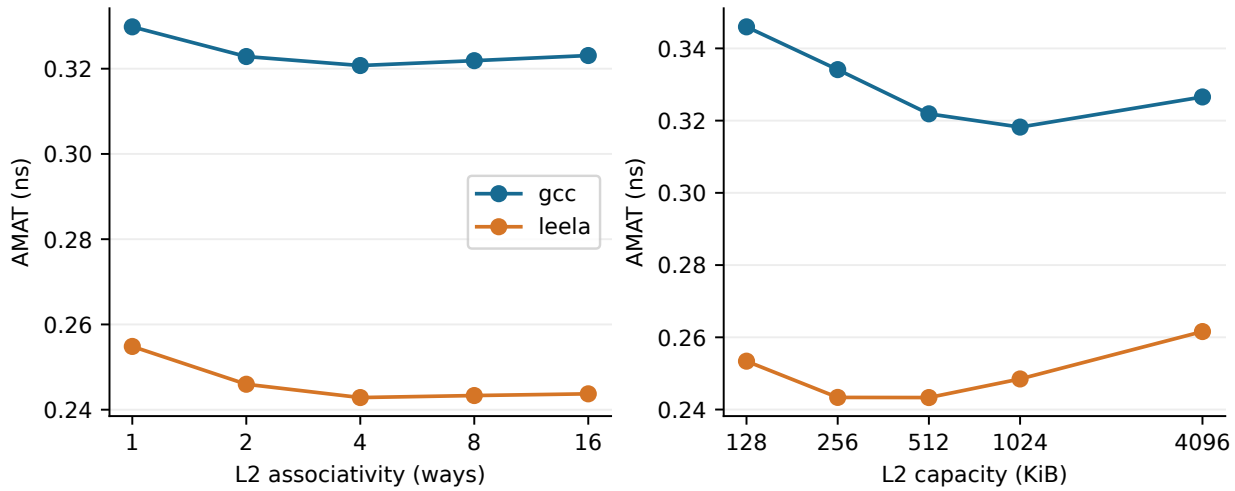


Figure 6: Controlled sensitivity experiments. Horizontal axes use base-2 logarithmic spacing with explicit measured ticks.

At 1, 2, 4, 8, and 16 L2 ways, mean AMAT is 0.29233, 0.28441, 0.28179, 0.28260, and 0.28340 ns. Four ways is best in this sweep, about 3.6% faster than one way. Beyond four ways, lookup cost offsets conflict-miss benefits. At 128, 256, 512, 1024, and 4096 KiB, mean AMAT is 0.29968, 0.28874, 0.28260, 0.28334, and 0.29407 ns. The minimum is 512 KiB. Growing to 4096 KiB raises AMAT by about 4.1%, area from 7.049 to 32.203 mm^2, and power from 148.72 to 436.70 mW. Filling the entire storage budget is not inherently beneficial.

6.1 Selected design

D12 changes the anchor's L2 block size to 128 bytes. Mean AMAT falls from 0.28260 to 0.25416 ns (10.1%), and area falls from 7.049 to 6.263 mm^2. Power rises from 148.72 to 152.72 mW (2.7%). Destiny optimizes internal organizations separately, so area need not vary monotonically with an architectural parameter. Lower AMAT wins despite slightly higher power.

7 Implementability: latency, power, and tag overhead

Destiny uses 2D SRAM at 65 nm, 350 K, the LOP setting, normal access, and optimization for read latency. These are model estimates, not fabricated-chip measurements; all configurations share these technology assumptions.

Table 5: Destiny metrics for D12: 8 KiB/64 B/4-way L1; 512 KiB/128 B/8-way L2.

Metric	L1	L2
Read-hit latency (ns)	0.2061	0.45478
Miss latency (ns)	0.18265	0.24966
Write latency (ns)	0.12703	0.22011
Read-hit energy (nJ)	0.066654	0.80579
Write energy (nJ)	0.061313	0.78358
Leakage (mW)	1.5043	57.814
Area (mm^2)	0.14333	6.1198
Tag-array area (mm^2)	0.039901	0.78592

Latency. L1 read-hit latency is 0.20610 ns, or 0.412 cycles at an illustrative 2 GHz. Rounding to at least one cycle is compatible with the few-cycle L1 target. L2 read-hit latency is 0.45478 ns. Core routing, arbitration, pipeline alignment, and controller delay are not fully represented; sub-cycle array estimates are not fractional-cycle architectural guarantees. They support plausibility, not timing closure.

Energy and power. For each level ℓ, the runner estimates

$$E_\ell = R_{h,\ell} e_{h,\ell} + W_{h,\ell} e_{w,\ell} + M_\ell (e_{m,\ell} + e_{w,\ell}), \tag{4}$$

including a fill write for each miss. Dividing $E_1 + E_2$ by L1 accesses gives demand energy per access. With energy in nJ and offered rate f in accesses/s,

$$P_{\text{dynamic}}[\text{mW}] = \overline{E}[\text{nJ/access}] f \, 10^{-6}. \tag{5}$$

At $f = 10^9$, 0.093404 nJ/access yields 93.404 mW dynamic power. Adding 59.319 mW leakage gives 152.722 mW. L1/L2 write energies are 0.061313/0.783585 nJ. Sub-nJ operations and about 0.153 W do not indicate obviously excessive cache-only demand, but no chip-wide budget was supplied. Dynamic power scales linearly with assumed activity; DRAM, writebacks, coherence, and replacement logic are excluded. Pin wall-clock time is not a hardware power measurement.

Metadata. For 64-bit addresses, $T = 64 - \log_2 B - \log_2 S$ tag bits. L1 has 128 lines, 32 sets, and 53-bit tags; L2 has 4096 lines, 512 sets, and 48-bit tags. With one valid bit per line and estimated LRU state of $A \log_2 A$ bits per set, overhead is 896 bytes (10.94%) for L1 and 26,624 bytes (5.08%) for L2: 27,520 bytes total. Dirty and coherence bits are excluded because they are not modeled. Destiny's tag-area fractions, 27.84% and 12.84%, also reflect peripheral circuits and differ from logical bit overhead. Even including estimated metadata, D12 uses only 560,000 bytes, below 5 MB.

8 Reproducibility and limitations

The repository supplies the simulator, `script.py`, `task4.py`, tests, and README. Prepare gcc/leela SPEC test workloads on Linux ECE and enter their input directories and argument arrays in `task4_config.json`:

```
python3 task4.py --check
python3 task4.py
```

The default `compact20` search is reported here; repeating the command resumes compatible completed jobs. Runs use private input copies. The committed export retains counters, commands, completion records, and Destiny inputs and outputs, but excludes licensed SPEC binaries and input copies. The manifest retains ECE paths for provenance; a TA must substitute locally available workloads. The exported run index uses relative paths for its saved statistics.

Rebuild this report's figures without rerunning benchmarks:

```
python3 -m pip install matplotlib
python3 report/build_figures.py
cd report
pdflatex main.tex
pdflatex main.tex
```

Tectonic is an alternative compiler. Corrected Tasks 1 and 2 measurements are saved under `results/task1-3-corrected/`. All six figures are vector PDFs with PNG previews, redrawn from numerical records rather than HTML screenshots.

8.1 Limits on conclusions

There is one run per benchmark/design and one random seed; small differences lack confidence intervals. Only test inputs are used, so train/ref inputs or other applications may select a different design. The sweep is not exhaustive, Destiny separately optimizes organizations, and equal benchmark weighting is a choice rather than an inherent objective. Simplified writes, virtual addresses, and omitted overlap limit real-processor claims. Counter consistency and exit status do not substitute for SPEC output-equivalence validation.

8.2 Corrected collection and verification

The corrected session is `session-1a552b2c84d04b334d9e` under `results/task1-3-corrected/`. Reproduce it with explicit gcc/leela commands in `lab123_config.json`:

```
python3 script.py --single-level-only --config lab123_config.json
```

The collection was resumed after interruption; completed run records were validated and reused. The final export contains 54 unique benchmark runs, 36 Task 1 rows and 24 Task 2 rows (six rows are shared), hardware models, commands, raw statistics, and per-task plots. Counter records are checked against completion markers, AMAT is recomputed, and the complete geometry/policy matrix is checked before export. The existing 40 two-level records support Task 3, along with the native cache tests. Task 4 retains its completed 20-design search. All requested experimental comparisons are covered within the declared model.

Sources

1. Course handout, `18344-f25-lab2-memoryhierarchy.pdf`, Tasks 1–4, pp. 1–3. The document itself is dated September 30, 2024.
2. `results/task4/session-3973386ea9622d3cee1b/`: manifest, measurement and hardware tables, design-space table, raw run records, and Destiny outputs.
3. `results/task1-3-corrected/`: corrected single-level measurements, hardware tables, raw run records, and validation summary.
4. Repository simulator and runner source; result-export commit `1243851`, compact-search runner commit `1facf8a`.